

03-12 Лекция: системный промпт, контекстное окно, стоимость токенов, function calling и резонинг в чат-ориентированных LLM

Дата и время: 2026-03-12 10:32:17

Локация: [Вставить локацию]

Инструктор: Секочёв

Резюме

Лекция от 2026-03-12 системно объясняет ключевые принципы работы чат-ориентированных языковых моделей: критическую роль системного промпта, диалоговый контекст и его ограничения, стоимость токенов, вызов функций (function calling) для расширения возможностей модели, а также технику резонинга (размышления перед ответом) для повышения качества решений без экспоненциального роста сложности моделей. Рассмотрены практические сценарии: диалоговое накопление контекста, управление контекстным окном и саммаризацией, экономические аспекты использования API, интеграция с файловой системой и внешними источниками (веб-поиск, коммерческий регистр), а также демо доступа к собственной LLM по адресу LLM.sekochev.ee.

Пункты знаний

1. Роль системного промпта

- Системный промпт определяет поведение модели
 - Системный промпт приходит первым и задаёт “как” модель отвечает: стиль, формат, роль, разрешённые/запрещённые действия; это важнее, чем параметры вроде температуры.
 - Пример: изменение системного промпта на “Отвечай только тремя словами” заставляет модель выдавать короткие ответы; без этого модель генерировала длинные.
- Определение роли и ограничений

- Через системный промпт можно задать роль модели, её возможности и запреты; это критично для специализированных задач, выходящих за рамки “средней” настройки обычного ChatGPT интерфейса.
- Стандартные чаты имеют преднастроенный системный промпт “для средней температуры по больнице”, но для специальных результатов нужно продумывать собственный системный промпт.

2. Модели, бренды и безопасность (согласование)

- Различие названий и сущностей
 - “ChatGPT” — маркетинговое имя одной из языковых моделей компании; это нарицательное название, подобно “Ferrari”, а моделей существует множество: Jenny, Mantroby, Claude (Anthropic) и др.
 - Архитектура и элементы работы моделей закодированы в их дизайне, чат — важная часть интерфейса.
- Безопасность и согласование (Alignment)
 - Модели имеют встроенные защитные установки, сформированные этапом fine-tuning; они отказываются давать вредоносные инструкции (например, рецепт бомбы), а также ограничивают ответы по чувствительным темам (биология, оружие, хакинг).
 - Разные модели по-разному применяют ограничения: пример с просьбой о рецепте “как словить для собаки” — модель отказала, ссылаясь на вред прикормки; некоторые продукты (упомянут скандал вокруг ответов на сексуальные вопросы у “игрока” Илона Маска) могут менее строго фильтровать.
 - Специализированные приложения (например, для Tinder-переписок) снимают ограничения в рамках подписки, тогда как общий ChatGPT может отказывать.

3. Диалог, контекст и контекстное окно

- Как поддерживается диалог
 - Модель “не помнит” прошлые сообщения; для продолжения диалога приложение каждый раз отправляет весь “бутерброд”: системный промпт, историю вопросов-ответов и текущий запрос.
 - Пример: вопрос “В чём смысл жизни?” → ответ “Смысл жизни в счастье” → следующий вопрос “А что такое счастье?” отправляется вместе с предыдущей историей.
- Ограничение контекстного окна и саммаризация

- Контекстное окно имеет максимальный размер (примерно 1,000,000 токенов в обсуждении). Если ответы длинные (например, по 50,000 токенов), то 20–40 обменов могут упереться в лимит.
- Решение: периодическая саммаризация (“сделай выжимку”) истории для сокращения контекста. Однако при сжатии возможна потеря важных деталей.
- Рекомендация: лучше “одна сессия — одна тема” для качества, хотя пользователи часто ведут очень длинные сессии.

4. Экономика токенов и стоимость использования

- Стоимость входящих/исходящих токенов
 - Примерные тарифы: 0.5 доллара за 1,000,000 входящих токенов и 5 долларов за 1,000,000 исходящих токенов (средняя модель).
 - При наивной реализации, когда на каждый запрос пересылается весь диалог, стоимость одного запроса может достигать ~0.5 доллара плюс генерация, что дорого при частых вызовах.
- Практическая нагрузка
 - Разработчики на больших кодовых базах могут тратить 600–800 тысяч евро, из-за огромных контекстов и множества итераций.
 - Агентные циклы (agent loop), многократные вызовы и длинные контексты резко увеличивают токен-потребление.

5. Вызов функций (Function Calling)

- Концепция и поток
 - Системный промпт может объявлять доступные функции (например, web-search). Модель, “не зная” актуальной информации, инициирует вызов функции.
 - Поток: системный промпт + пользовательский запрос → модель решает вызвать функцию → приложение перехватывает вызов → выполняет функцию (например, “погода в Таллине”) → возвращает результат (“+15, солнечно”) → весь контекст (включая результат функции) снова подаётся модели → модель формирует финальный ответ пользователю.
- Популярные функции и интеграции
 - Работа с файлами: список файлов, открыть, прочитать, изменить — позволяет агенту взаимодействовать с локальной файловой системой, чтобы не копипастить документы в чат.

- Доступ к базам данных и реестрам: пример интеграции с коммерческим регистром для выборок по адресам и характеристикам (товарищества квартир в Лас-На-Мяэ, дома старше 1990 года).
- Внешние запросы: веб-поиск для актуализации знаний моделей, обученных на снимке данных (например, снимок января 2024, тренировка 8 месяцев, готовность декабрь 2025, без знания событий между).
- Нагрузка на контекст
 - Каждый вызов функции увеличивает “бутерброд” контекста, потому что для смысла диалога нужно заново передавать всю историю, системный промпт, вызов и ответ функции. Это сильно увеличивает число токенов и стоимость.

6. Резонинг (размышления перед ответом)

- Проблема “без бэкспейса”
 - Генерация ответов идёт токен за токеном, без возможности “откатывать” начальный выбор. Неподходящий первый токен может “зафиксировать” плохую траекторию ответа.
- Решение
 - Разрешить модели “подумать” на черновике до ответа: анализ намерения пользователя, уточнение понятий, разбор вариантов, проверка упущений (“а что если я не права? что упускаю?”).
 - Эффект: кратный рост качества ответов без необходимости делать модель огромной. При небольшом увеличении времени (например, минута на размышления) можно использовать значительно более дешёвое железо (машина за 5k вместо 100k долларов).
 - Практика: в продвинутых моделях этап резонинга скрыт за спиннером, но его можно наблюдать в открытом режиме; он объясняет разницу между длинным “черновым мышлением” и коротким выводом (пример оценки 50,000 токенов для размышлений).
- Сценарии применения
 - Письмо по email, составление документов, решение олимпиадных задач: сначала безопасный черновик, поиск тупиковых путей, затем аккуратный ответ.
 - Прямое улучшение точности, глубины анализа и устойчивости к неоднозначным запросам (“полная ерунда” — прояснение, что именно имелось в виду).

7. Агенты и локальные интеграции

- Агент как “исполнитель”
 - Модель — “мозг в банке” без действий; агент на вашем компьютере выполняет функции (работает с файлами, базами, вебом) по командам модели через function calling.
 - Пользовательская логика: системный запрос инструктирует модель запрашивать у агента списки файлов, содержимое, изменения; агент выполняет быстро на локальном хосте, модель принимает результаты и формирует ответы.
- Практические цели
 - Устранение ручного копипаста; автоматизация работы с договорами, реквизитами, изменениями цен/товаров; доступ к собственным корпоративным источникам.

8. Демонстрация и доступ к модели

- Демо доступа
 - Инструктор поднял языковую модель на своём сервере с двумя видеокартами, включающуюся по запросу; предложен доступ по адресу LLM.sekochev.ee.
 - Показана работа: приветственное сообщение, подключение, автоподстановка названия модели, масштабирование интерфейса.

Вопросы

- [Insert Question/Confusion]

Следующие шаги

- [] Сформулировать и отредактировать собственный системный промпт для специализированной задачи, включая стиль, формат ответов и ограничения.
- [] Настроить стратегию управления контекстом: периодическая саммаризация длинных диалогов, правило “одна сессия — одна тема”.
- [] Рассчитать ориентировочную стоимость использования модели в своём сценарии: оценить входящие/исходящие токены на запрос, частоту вызовов и оптимизации для снижения затрат.

- [] Реализовать function calling для как минимум двух функций: веб-поиск актуальной информации и работа с локальной файловой системой (список/читать/изменить файл).
- [] Интегрировать доступ к внешнему источнику данных (например, коммерческий регистр) и разработать пример запроса с фильтрами (район, год постройки).
- [] Включить этап резонинга в пайплайн ответа: предоставить модели “черновое мышление” перед выдачей результата; задать эвристики (“что я упускаю?”).
- [] Настроить локальный агент на ПК для выполнения функций по командам модели, проверить скорость и корректность работы.
- [] Пройти по адресу [LLM.sekochev.ee](https://llm.sekochev.ee) и протестировать подключение к модели, зафиксировать поведение, название модели и оценить интерфейс.